Міністерство освіти і науки України

Львівський національний університет імені Івана Франка

Факультет електроніки та комп'ютерних технологій

ЗВІТ

про виконання лабораторної роботи №4

“ Професійне ООП та система типів Python”

Виконала:

Студентка групи ФЕП-33с

Най Анастасія

Перевірив:

Проф. Свелеба С.А.

Львів-2026

Мета роботи: сформувати практичні навички побудови професійної об'єктно-орієнтованої моделі в Python із використанням класів, композиції, наслідування, абстрактних базових класів, протоколів, dataclass, properties, dunder methods, принципів SOLID та сучасної системи type hints.

Варіант 10. Система обліку складських запасів

Domain objects:

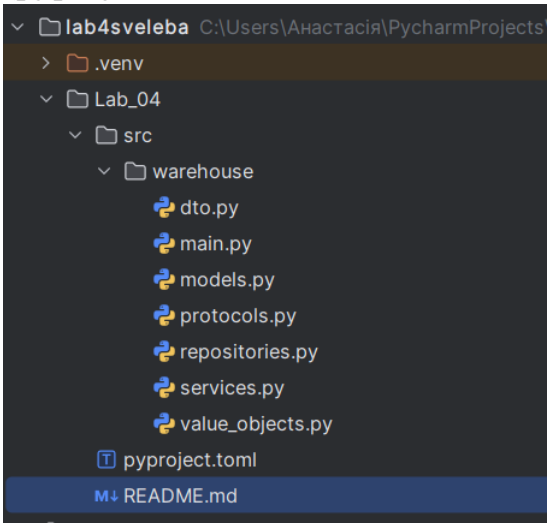
- Product;
- WarehouseItem;
- Warehouse;
- Supplier;
- StockOperation.

Реалізувати:

- receipt;
- write-off;
- пошук товару за кодом;
- розрахунок загальної вартості складу;
- low-stock rule;
- repository;
- reorder policy і report service.

Хід роботи:

1. Спочатку я створила робочу папку проєкту. Організувала професійну структуру за стандартом src-layout: у папці src зосередила весь програмний код, залишивши корінь проєкту для конфігураційних файлів (pyproject.toml, README.md).



2. У файлі pyproject.toml я описала конфігурацію проєкту (warehouse_project), мінімальну версію Python, налаштування збірки з використанням setuptools, а також додала секцію конфігурації для суворого статичного аналізатора типів mypy (strict = true, explicit_package_bases = true).

```toml
[build-system]
requires = ["setuptools>=61.0"]
build-backend = "setuptools.build_meta"

[project]
name = "warehouse_project"
version = "0.1.0"
dependencies = []

[tool.mypy]
python_version = "3.12"
strict = true
warn_return_any = true
warn_unused_ignores = true
explicit_package_bases = true
```

3. У файлі src/warehouse/value_objects.py я реалізувала незмінний об'єкт-значення Money як незмінний frozen dataclass із використанням slots, додала валідацію на заборону від'ємних сум у методі __post_init__ та перевантажила арифметичний dunder-метод __add__ для безпечного додавання грошових значень з перевіркою валют.

```python
from dataclasses import dataclass

@dataclass(frozen=True, slots=True)
class Money:
    amount: float
    currency: str = "UAH"

    def __post_init__(self) -> None:
        if self.amount < 0:
            raise ValueError("Money amount cannot be negative")

    def __add__(self, other: "Money") -> "Money":
        if self.currency != other.currency:
            raise ValueError("Currency mismatch")
        return Money(self.amount + other.amount, self.currency)

    def __str__(self) -> str:
        return f"{self.amount:.2f} {self.currency}"
```

4. У файлі src/warehouse/models.py я створила основні доменні сутності (Supplier, Product, WarehouseItem, Warehouse), застосувала інкапсуляцію через @property (розрахунок загальної вартості позиції та прапорець низького залишку is_low_stock), спроєктувала абстрактний базовий клас StockOperation (ABC) з конкретними дочірніми реалізаціями операцій надходження (ReceiptOperation) та списання (WriteOffOperation), а також реалізувала клас-контейнер Warehouse із підтримкою dunder-методів (__len__, __iter__, __contains__).

```python
from collections.abc import Iterator
from abc import ABC, abstractmethod
from dataclasses import dataclass, field
from .value_objects import Money

@dataclass(frozen=True)
class Supplier:
    id: int
    name: str
    contact_email: str

@dataclass
class Product:
    code: str
    name: str
    _price: Money
    supplier: Supplier

    @property
    def price(self) -> Money:
        return self._price

    def __str__(self) -> str:
        return f"{self.name} ({self.code}) - {self._price}"

@dataclass
class WarehouseItem:
    product: Product
    quantity: int = 0
    min_threshold: int = 5

    def __post_init__(self) -> None:
        if self.quantity < 0:
            raise ValueError("Quantity cannot be negative")

    @property
    def total_value(self) -> Money:
        return Money(self.product.price.amount * self.quantity,
self.product.price.currency)

    @property
    def is_low_stock(self) -> bool:
        return self.quantity < self.min_threshold

# Абстрактний базовий клас для складських операцій (ABC)
class StockOperation(ABC):
    @abstractmethod
    def execute(self, item: WarehouseItem) -> None:
        pass

class ReceiptOperation(StockOperation):
    def __init__(self, qty: int) -> None:
        self.qty = qty

    def execute(self, item: WarehouseItem) -> None:
        if self.qty <= 0:
            raise ValueError("Receipt quantity must be positive")
        item.quantity += self.qty

class WriteOffOperation(StockOperation):
    def __init__(self, qty: int) -> None:
        self.qty = qty

    def execute(self, item: WarehouseItem) -> None:
        if self.qty <= 0:
            raise ValueError("Write-off quantity must be positive")
```

```
        if item.quantity < self.qty:
            raise ValueError("Insufficient stock for write-off")
        item.quantity -= self.qty

# Склад як контейнер із підтримкою dunder methods
@dataclass
class Warehouse:
    id: int
    name: str
    items: list[WarehouseItem] = field(default_factory=list)

    def add_item(self, item: WarehouseItem) -> None:
        self.items.append(item)

    def find_by_code(self, code: str) -> WarehouseItem | None:
        for item in self.items:
            if item.product.code == code:
                return item
        return None

    def __len__(self) -> int:
        return len(self.items)

    def __iter__(self) -> Iterator[WarehouseItem]:
        return iter(self.items)

    def __contains__(self, code: str) -> bool:
        return any(item.product.code == code for item in self.items)

    @property
    def total_inventory_value(self) -> Money:
        total = sum(item.total_value.amount for item in self.items)
        currency = self.items[0].product.price.currency if self.items else "UAH"
        return Money(total, currency)
```

5. У файлі src/warehouse/protocols.py я описала структурні інтерфейси за допомогою механізму Protocol (зокрема Notifier та ReorderPolicy) з декоратором @runtime_checkable, що дозволяє забезпечити поліморфізм та кауплинг без жорсткого наслідування.

```
from typing import Protocol, runtime_checkable
from .models import WarehouseItem

@runtime_checkable
class Notifier(Protocol):
    def send(self, recipient: str, message: str) -> None:
        ...

@runtime_checkable
class ReorderPolicy(Protocol):
    def check_and_reorder(self, item: WarehouseItem, notifier: Notifier) ->
None:
        ...
```

6. У файлі src/warehouse/dto.py я визначила структуру зовнішніх даних (ProductPayload) на основі TypedDict для типізації словників та коректного мапування даних із зовнішніх джерел чи API у доменні об'єкти.

```
from typing import TypedDict

class ProductPayload(TypedDict):
    code: str
    name: str
    price: float
    currency: str
    supplier_id: int
    supplier_name: str
    supplier_email: str
```

7. У файлі src/warehouse/repositories.py я розробила універсальний дженерик-репозиторій InMemoryRepository, використавши Generic[T] та обмежений TypeVar (bound=HasCode), що дозволяє повторно використовувати логіку зберігання для будь-яких доменних сутностей із підтримкою CRUD-операцій у пам'яті.

```
from typing import Generic, Protocol, TypeVar

class HasCode(Protocol):
    code: str

T = TypeVar("T", bound=HasCode)

class InMemoryRepository(Generic[T]):
    def __init__(self) -> None:
        self._items: dict[str, T] = {}

    def add(self, item: T) -> None:
        self._items[item.code] = item

    def get(self, code: str) -> T | None:
        return self._items.get(code)

    def all(self) -> list[T]:
        return list(self._items.values())

    def remove(self, code: str) -> bool:
        return self._items.pop(code, None) is not None
```

8. У файлі src/warehouse/services.py я реалізувала сервіс бізнес-логіки WarehouseService та політику замовлень StandardReorderPolicy, застосувавши принцип ін'єкції залежностей (Dependency Injection) через передачу абстракцій (Notifier, ReorderPolicy) у конструктор, що повністю задовольняє принципам SOLID (DIP та SRP).

```
from .models import Warehouse, WarehouseItem, StockOperation
from .protocols import Notifier, ReorderPolicy
from .value_objects import Money


class StandardReorderPolicy:
    def check_and_reorder(self, item: WarehouseItem, notifier: Notifier) ->
None:
        if item.is_low_stock:
            notifier.send(
                item.product.supplier.contact_email,
                f"Low stock alert for {item.product.name} (Code:
{item.product.code}). Current quantity: {item.quantity}."
```

```
            )


class WarehouseService:
    def __init__(self, warehouse: Warehouse, notifier: Notifier, reorder_policy:
ReorderPolicy) -> None:
        self._warehouse = warehouse
        self._notifier = notifier
        self._reorder_policy = reorder_policy

    def perform_operation(self, code: str, operation: StockOperation) -> None:
        item = self._warehouse.find_by_code(code)
        if not item:
            raise ValueError(f"Product with code {code} not found in
warehouse.")

        operation.execute(item)
        self._reorder_policy.check_and_reorder(item, self._notifier)

    def get_total_value(self) -> Money:
        return self._warehouse.total_inventory_value
```

9. У файлі src/warehouse/main.py я написала демонстраційний сценарій (точку входу), який ініціалізує початкові дані (постачальників, товари, складські позиції), виконує складські операції надходження та списання, перевіряє спрацювання правил контролю низьких залишків (low-stock rule) і виводить результати роботи всієї типізованої об'єктної моделі в консоль.

```
from src.warehouse.models import Warehouse, WarehouseItem, Product, Supplier,
ReceiptOperation, WriteOffOperation
from src.warehouse.value_objects import Money
from src.warehouse.services import WarehouseService, StandardReorderPolicy


class ConsoleNotifier:
    def send(self, recipient: str, message: str) -> None:
        print(f"[NOTIFICATION sent to {recipient}]: {message}")


def main() -> None:
    supplier = Supplier(id=1, name="TechCorp Ltd",
contact_email="orders@techcorp.com")

    laptop = Product(code="P-001", name="Gaming Laptop", _price=Money(45000.0,
"UAH"), supplier=supplier)
    mouse = Product(code="P-002", name="Wireless Mouse", _price=Money(1200.0,
"UAH"), supplier=supplier)

    item1 = WarehouseItem(product=laptop, quantity=4, min_threshold=5)  # нижче
порогового значення
    item2 = WarehouseItem(product=mouse, quantity=25, min_threshold=10)

    warehouse = Warehouse(id=101, name="Main Central Warehouse")
    warehouse.add_item(item1)
    warehouse.add_item(item2)

    print(f"Загальна кількість товарних позицій на складі: {len(warehouse)}")
    print(f"Чи є товар 'P-001' на складі? {'P-001' in warehouse}")
    print(f"Початкова загальна вартість складу:
{warehouse.total_inventory_value}")
```

```
    notifier = ConsoleNotifier()
    reorder_policy = StandardReorderPolicy()
    service = WarehouseService(warehouse=warehouse, notifier=notifier,
reorder_policy=reorder_policy)

    print("\n--- Виконання операції надходження (Receipt) для миші ---")
    service.perform_operation("P-002", ReceiptOperation(10))

    print("\n--- Виконання операції списання (Write-Off) для ноутбука
(спрацьовує low-stock rule) ---")
    service.perform_operation("P-001", WriteOffOperation(1))

    print(f"\nКінцева загальна вартість складу: {service.get_total_value()}")


if __name__ == "__main__":
    main()
```

10. У файлі README.md я оформила повну документацію до проєкту, де описала мету лабораторної роботи, індивідуальний варіант (Система обліку складських запасів), структуру папок та файлів, реалізовані принципи ООП і SOLID.

```
# Лабораторна робота №4: Професійне ООП та система типів Python

**Система обліку складських запасів (Варіант 10)**

## Мета роботи
Сформувати практичні навички побудови професійної об'єктно-орієнтованої моделі в
Python із використанням класів, композиції, наслідування, абстрактних базових
класів (ABC), протоколів (Protocol), `dataclass`, властивостей (`properties`),
dunder-методів, принципів SOLID та сучасної системи типів (`type hints`,
`Generic`, `TypeVar`, `TypedDict`).

---

## Архітектура та структура проєкту

Проєкт організовано відповідно до сучасних стандартів проєктування модульних
Python-додатків:

```text
warehouse_project/
│
├── pyproject.toml          # Конфігурація проєкту та налаштування муру
├── README.md               # Документація проєкту
└── src/
    └── warehouse/
        ├── __init__.py
        ├── value_objects.py  # Незмінні об'єкти-значення (frozen dataclass)
        ├── models.py         # Доменні сутності, ABC-операції, контейнери
        ├── protocols.py      # Структурні інтерфейси (Protocol)
        ├── dto.py            # Зовнішні структури даних (TypedDict)
        ├── repositories.py   # Універсальний дженерик-репозиторій
        ├── services.py       # Бізнес-логіка та ін'єкція залежностей (DI)
        └── main.py           # Точка входу та демонстраційний сценарій
```
```

Результати:

```
Загальна кількість товарних позицій на складі: 2
Чи є товар 'P-001' на складі? True
Початкова загальна вартість складу: 210000.00 UAH

--- Виконання операції надходження (Receipt) для миші ---

--- Виконання операції списання (Write-Off) для ноутбука (спрацьовує low-stock rule) ---
[NOTIFICATION sent to orders@techcorp.com]: Low stock alert for Gaming Laptop (Code: P-001). Current quantity: 3.

Кінцева загальна вартість складу: 177000.00 UAH

Process finished with exit code 0
```

```
(.venv) PS C:\Users\Анастасія\PycharmProjects\lab4sveleba> mypy src
Success: no issues found in 7 source files
```

Висновок: під час виконання лабораторної роботи я успішно перейшла від процедурного програмування до побудови повноцінної, типізованої об'єктно-орієнтованої моделі предметної області для системи обліку складських запасів. Завдяки використанню dataclass, frozen value objects, властивостей, наслідування та композиції було створено гнучку і структуровану архітектуру. Застосування абстрактних базових класів (ABC), протоколів (Protocol), дженериків (Generic, TypeVar) та ін'єкції залежностей дозволило повністю дотриматися принципів SOLID, забезпечивши низький рівень зв'язності та високу розширюваність коду. Крім того, інтеграція сучасної системи типів (type hints, TypedDict) та проходження статичної перевірки за допомогою mypy дали змогу виявити потенційні помилки ще на етапі розробки до запуску програми.